

Lab2 实验报告

潘文峥
520030910232

1.实验目的

- 1. 掌握单周期 CPU 各个控制信号的作用和生成过程。
- 2. 掌握单周期 CPU 控制器的工作原理及其设计方法。
- 3. 理解单周期 CPU 执行指令的过程。
- 4. 掌握取指、译码阶段数据通路、控制器的执行过程。
- 5. 掌握RISC-V 架构下不同指令的立即数的构成和立即数产生模块的设计方法。

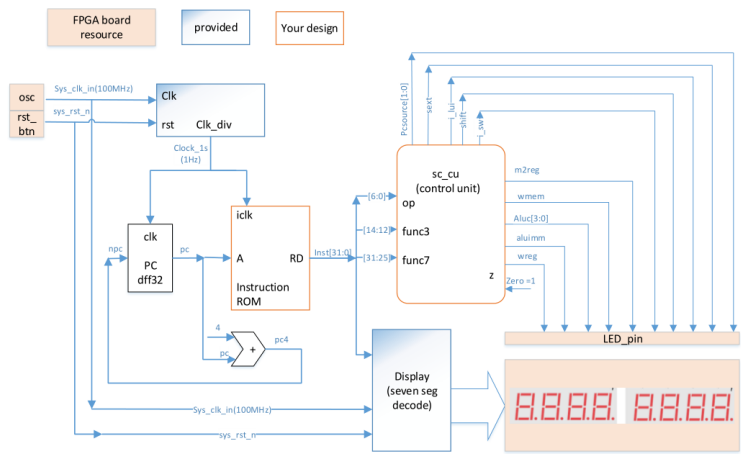
2.实验平台和器材

- 1. 计算机1台（尽可能达到 8G 及以上内存）；
- 2. EGO1实验开发板；
- 3. Xilinx Vivado 开发套件（2020.2 版本）。

3.实验任务

3.1 存储器与控制器设计与验证实验任务

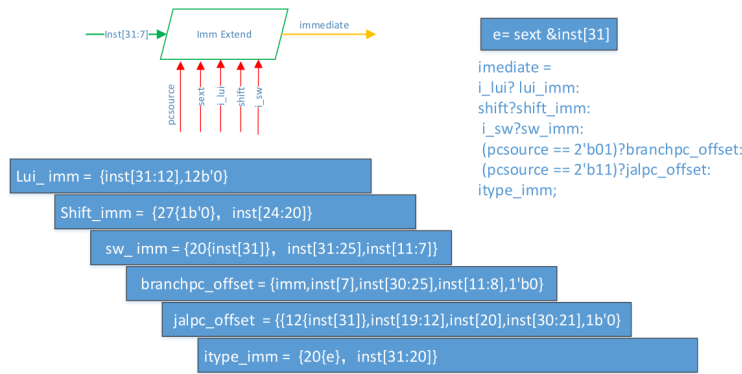
- 依据取指、译码阶段的需求，需要在实验一基础上实现以下模块:



3.2 立即数产生模块实验任务

- RISC-V架构指令中含有立即数操作的不同类型指令的机器码中立即数构成方式不尽相同.本模块用于区分支持不同类型的指令时，产生所需要的经有符号扩展后的32bit立即数，用于后续ALU模块的计算或者对

下一条指令的PC值更新。

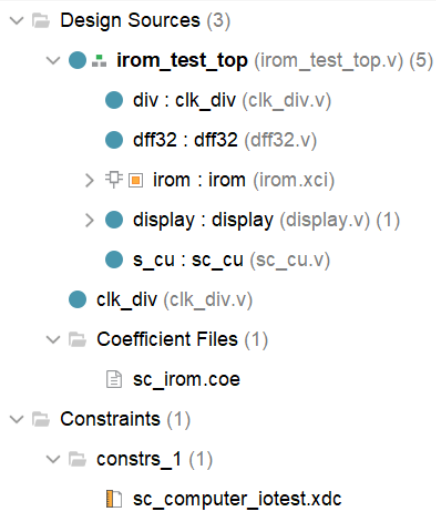


4.任务实现与结果演示

4.1 CPU控制器模块设计

1. 文件树:

- 导入了clk_div.v、dff32.v、display.v、sc_computer_iotest.xdc 预置文件，实现了sc_cu.v控制模块、irom_tes_top.v顶层设计模块，导入了在Lab2-1中完成的指令文件sc_irom.coe。如下图所示。



2. 关键代码:

- 控制模块，主要根据传入的32为指令码完成译码，先确定指令类型，再根据指令具体操作给相应的控制信号赋值。

```
module sc_cu (inst, z, wmem, wreg, m2reg, aluc, aluimm, pcsource,
sext,i_lui,i_sw,shift);
    input  [31:0] inst;
    input      z;
    output      wreg,m2reg,aluimm,sext,wmem,i_lui,i_sw,shift;
    output [3:0] aluc;
    output [1:0] pcsource;
    wire [6:0] op = inst[6:0];
    wire [2:0] func3 = inst[14:12];
```

```

wire [6:0] func7 = inst[31:25];

wire r_type = (op == 7'b0110011); //首先根据op码确定指令类型
wire i_type = (op == 7'b0010011);

wire i_add = r_type & (func3 == 3'b000) & ~inst[30]; //R type
wire i_sub = r_type & (func3 == 3'b000) & inst[30];
wire i_and = r_type & (func3 == 3'b111);
//.....
wire i_addi = i_type & (func3 == 3'b000 ); //I type
wire i_srli = i_type & (func3 == 3'b101 ) & ~inst[30];
wire i_lw = (op == 7'b0000011) & (func3 == 3'b010);
//.....
wire i_sw = (op == 7'b0100011) & (func3 == 3'b010); //S type
wire i_beq = (op == 7'b1100111) & (func3 == 3'b000); //SB type
wire i_bne = (op == 7'b1100111) & (func3 == 3'b001); //U type
wire i_lui = (op == 7'b0110111);
wire i_jal = (op == 7'b1101111); //UJ type

//根据指令操作类型，给对应控制信号赋值
assign pcsource[1] = i_jalr | i_jal;
assign pcsource[0] = ( i_beq & z ) | (i_bne & ~z) | i_jal ;
assign sext = i_addi | i_lw | i_jalr | i_sw | i_beq | i_bne | i_jal;
assign wmem = i_sw;
assign m2reg = i_lw;
assign shift = i_sll | i_srl | i_sra | i_slli | i_srli | i_srai ;
assign aluc[3] = i_sub | i_sra | i_srai | i_beq | i_bne ;
//.....

endmodule

```

- 顶层调用模块

```

module irom_test_top(
    input sys_rst_n,
    input sys_clk_in,
    input [4:0] sw_pin,
    input [4:0] dip_pin,
    output [7:0] seg_data_0_pin, //output A1,B1,C1,D1,E1,F1,G1,DP1,
    output [7:0] seg_data_1_pin, //output A0,B0,C0,D0,E0,F0,G0,DP0,
    output [7:0] seg_cs_pin,
    //DN1_K4,DN1_K3,DN1_K2,DN1_K0,DN0_K4,DN0_K3,DN0_K2,DN0_K1
    output [0:15] led_pin
);
wire [31:0] pc, inst, pc4;
wire clock_1s;

wire wreg,m2reg,aluimm,sext,wmem,i_lui,i_sw,shift,z;
wire [3:0] aluc;
wire [1:0] pcsource;

```

```

clk_div div(          //实现分频, 获得clock_1s信号
.clkin(sys_clk_in),
.clkout(clock_1s)
);

assign pc4 = pc + 32'h4; //每周期将pc地址加4

dff32 dff32(          //D触发器, 每周期将新的pc输出到irom
.clk(clock_1s),
.clrn(sys_rst_n),
.d(pc4),
.q(pc)
);

irom irom(
.clka(clock_1s),      // 输入时钟信号
.addra(pc[7:2]),      // 输入当前指令地址pc
.douta(inst)         // 输出当前指令编码
);

display display(
.clk(sys_clk_in),
.reset(sys_rst_n),
.s(inst),             //将当前的32位指令码通过七段数码管显示 (十六进制)
.seg0(seg_data_0_pin),
.seg1(seg_data_1_pin),
.ans(seg_cs_pin)
);

sc_cu s_cu(
.inst(inst),          //输入当前指令
.z(1'b1),
.wreg(wreg),          //以下为各个控制信号输出
.m2reg(m2reg),
.aluimm(aluimm),
.sext(sext),
.wmem(wmem),
.i_lui(i_lui),
.i_sw(i_sw),
.shift(shift),
.aluc(aluc),
.pcsource(pcsource)
);

//通过板上LED显示各控制信号情况
assign led_pin = {inst[3:0], z, wmem, wreg, m2reg, aluc, aluimm,
pcsource[1:0], sext, i_lui, i_sw, shift};

endmodule

```

4.2 立即数产生模块设计

根据sc_cu模块来的控制信号`sxt`, `i_lui`, `i_sw`, `shift`, `pcsource[1:0]`, 依据不同类型的指令, 将对应类型指令机器码 `inst[31:0]` 中的立即数信息位, 按照该指令定义重新排列组合, 并考虑符号扩展后输出, 形成32bit的 `out_immediate` 信号。

```
module immext(inst,pcsource,sxt,i_lui,i_sw,shift,out_immediate);
    input [31:0] inst;
    input sxt,i_lui,i_sw,shift;
    input [1:0] pcsource;
    output [31:0] out_immediate;

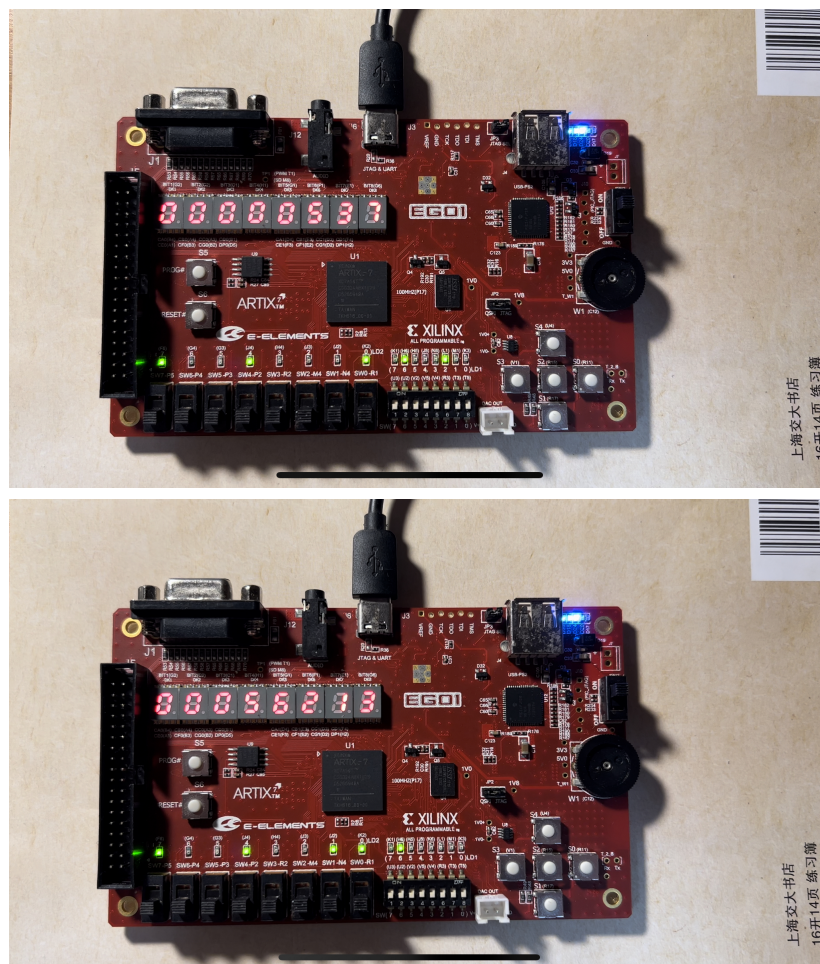
    wire e = sxt & inst[31];

    wire [31:0] shift_imm = {27'b0,inst[24:20]};
    wire [31:0] sw_imm = {{20{inst[31]}},inst[31:25],inst[11:7]};
    wire [31:0] branchpc_offset = {12'b0,inst[7],inst[30:25],inst[11:8],1'b0};
    wire [31:0] jalpc_offset =
    {{12{inst[31]}},inst[19:12],inst[20],inst[30:21],1'b0};
    wire [31:0] itype_imm = {{20{e}},inst[31:20]};
    wire [31:0] lui_imm = {inst[31:12],12'b0};

    assign out_immediate = i_lui ?  lui_imm:
                           shift ?  shift_imm:
                           i_sw ?   sw_imm:
                           (pcsource == 2'b01) ?  branchpc_offset:
                           (pcsource == 2'b11) ?  jalpc_offset:
                           itype_imm;

endmodule
```

4.3 控制器板级验证截图



如图，板上将以约1s的频率更新指令，数码管部分显示当前取得的指令代码（以16进制表示），下方LED部分展示对应控制信号（如`aluc`、`m2reg`等）的状态。

5.归纳总结

- 通过本次实验，我对单周期 CPU 各个控制信号的作用和生成过程有了更深入的理解，并能通过Verilog语言设计实现单周期 CPU 控制器的工作原理。在实现过程中，我对取指、译码阶段数据通路、控制器的执行过程更加熟悉和掌握了。另外，RISC-V 架构下不同指令的立即数的构成和立即数产生模块的设计方法也得到了巩固。
- 另外，通过在实验中遇到问题、分析问题，与同学讨论并解决问题的过程，使我的动手能力和处理问题思维得到了锻炼和提高，对理论到实践的工程特点有了进一步的理解。